BRAC University **SET A**

Department of Computer Science and Engineering

CSE111: Programming Language II

Examination: *Quiz #4* Semester: *Summer 2022*

Date: 29 / 08 / 2022 Time: 40 Minutes

ID:	Name:	Obtained Points:	Section:
_ _ _ _ _ _ _ _	*(Please write in **CAPITAL LETTERS**)*	/ 20	**42**

- Read questions carefully.
- Understanding the question is part of the exam, please do not ask questions.

Answer to Question 1

1. Design the **CSE** and **EEE** classes derived from **Department** class so that the following output is generated upon running the given driver code. [**CO2**, **CO4**, **CO5**] (**20 Marks**)

<table>
<tr><th>Driver code (with Department class)</th><th>Expected output</th></tr>
<tr><td>

```
class Department:
    objects = []
    def __init__(self, semester, dep=None):
        self.semester = semester
        self.dep = dep
        self.name = "Default"
        self.sid = -1
        self.approved = []
        Department.objects.append(self)

    def __repr__(self):
        return f"{self.dep}({self.name})"

    def student_info(self):
        print("Name:", self.name)
        print("ID:", self.sid)
        print("Department:", self.dep)
        print("Approved:", self.approved)

    def approve(self, *courses):
        print("No courses Approved yet!")

# Design CSE and EEE classes

s1 = CSE("Rahim", 16101328, "Spring2016")
s1.student_info()
print("------------------")
s1.approve("CSE110", "MAT110", "ENG101")
print("------------------")
s1.approve("CSE110", "MAT120", "ENG102")
print("==================")
s2 = EEE("Tanzim", 18101326, "Spring 2018")
s2.student_info()
print("------------------")
s2.approve("Mat110", "PHY111", "ENG101")
print("==================")
s2.student_info()
print("==================")
print(Department.objects)
print(CSE.objects)
print(EEE.objects)
```

</td><td>

```
Name: Rahim
ID: 16101328
Department: CSE
Approved: []
------------------
Approved: CSE110, MAT110, ENG101
------------------
Approved: MAT120, ENG102
Dismissed: CSE110
==================
Name: Tanzim
ID: 18101326
Department: EEE
Approved: []
------------------
Approved: Mat110, PHY111, ENG101
==================
Name: Tanzim
ID: 18101326
Department: EEE
Approved: ['Mat110', 'PHY111', 'ENG101']
==================
[CSE(Rahim), EEE(Tanzim)]
[CSE(Rahim)]
[EEE(Tanzim)]
```

</td></tr>
</table>

BRAC University SET B

Department of Computer Science and Engineering

CSE111: Programming Language II

Examination: *Quiz #4* Semester: *Summer 2022*

Date: 29 / 08 / 2022 Time: 40 Minutes

ID:	Name:	Obtained Points:	Section:
_ _ _ _ _ _ _ _	*(Please write in **CAPITAL LETTERS**)*	/ 20	**42**

- Read questions carefully.
- Understanding the question is part of the exam, please do not ask questions.

<u>Answer to Question 1</u>

1. Design the **Agora** and **Shwapno** classes derived from **GroceryShop** class so that the following output is generated upon running the given driver code. [**CO2**, **CO4**, **CO5**] (**20 Marks**)

Driver code (with **GroceryShop** class)	Expected output

Driver code (with **GroceryShop** class):

```
class GroceryShop:
    objects = []
    def __init__(self, platform, shop=None):
        self.platform = platform
        self.shop = shop
        self.name = "Default"
        self.cid = -1
        self.cart = []
        GroceryShop.objects.append(self)

    def __repr__(self):
        return f"{self.shop}({self.name})"

    def customer_info(self):
        print("Name:", self.name)
        print("CID:", self.cid)
        print("Shop:", self.shop)
        print("Cart:", self.cart)

    def buy(self, *items):
        print("No items bought yet!")

# Design the Agora and Shwapno classes


c1 = Agora("Rahim", 16101328, "In-store")
c1.customer_info()
print("------------------")
c1.buy("Pepsi", "Chips", "Water")
print("------------------")
c1.buy("Juice", "Pepsi")
print("==================")
c2 = Shwapno("Tanzim", 18101326, "Online")
c2.customer_info()
print("------------------")
c2.buy("Mat110", "PHY111", "ENG101")
print("==================")
c2.customer_info()
print("==================")
print(GroceryShop.objects)
print(Agora.objects)
print(Shwapno.objects)
```

Expected output:

```
Name: Rahim
CID: 16101328
Shop: Agora
Cart: []
------------------
Added: Pepsi, Chips, Water
------------------
Added: Juice
Discarded: Pepsi
==================
Name: Tanzim
CID: 18101326
Shop: Shwapno
Cart: []
------------------
Added: Mat110, PHY111, ENG101
==================
Name: Tanzim
CID: 18101326
Shop: Shwapno
Cart: ['Mat110', 'PHY111', 'ENG101']
==================
[Agora(Rahim), Shwapno(Tanzim)]
[Agora(Rahim)]
[Shwapno(Tanzim)]
```

BRAC University SET C

Department of Computer Science and Engineering

CSE111: Programming Language II

Examination: *Quiz #4* Semester: *Summer 2022*
Date: 29 / 08 / 2022 Time: 40 Minutes

ID:	Name:	Obtained Points:	Section:
_ _ _ _ _ _ _ _	*(Please write in **CAPITAL LETTERS**)*	/ 20	**42**

- Read questions carefully.
- Understanding the question is part of the exam, please do not ask questions.

Answer to Question 1

1. Design the **Bangladeshi** and **Japanese** classes derived from **Nationality** class so that the following output is generated upon running the given driver code. [**CO2**, **CO4**, **CO5**] (**20 Marks**)

Driver code (with **Nationality** class)	Expected output

class Nationality:
    objects = []
    def __init__(self, gender, nationality=None):
        self.gender = gender
        self.nationality = nationality
        self.name = "Default"
        self.nid = -1
        self.skills = []
        Nationality.objects.append(self)
 	
    def __repr__(self):
        return f"{self.nationality}({self.name})"
 	
    def citizen_info(self):
        print("Name:", self.name)
        print("NID:", self.nid)
        print("Nationality:", self.nationality)
        print("Skills:", self.skills)
 	
    def addSkills(self, *skills):
        print("No skills!")
 	
# Design Bangladeshi and Japanese classes 	
 	
p1 = Bangladeshi("Rahim", 16101328,
"Bangladeshi")
p1.citizen_info()
print("------------------")
p1.addSkills("CSE110", "MAT110", "ENG101")
print("------------------")
p1.addSkills("CSE110", "MAT120", "ENG102")
print("==================")
p2 = Japanese("Tanzim", 18101326, "Japanese")
p2.citizen_info()
print("------------------")
p2.addSkills("Mat110", "PHY111", "ENG101")
print("==================")
p2.citizen_info()
print("==================")
print(Nationality.objects)
print(Bangladeshi.objects)
print(Japanese.objects)
	Name: Rahim
	NID: 16101328
	Nationality: Bangladeshi
	Skills: []
	------------------ 	
	Skills: CSE110, MAT110, ENG101
	------------------ 	
	Skills: MAT120, ENG102
	Duplicates: CSE110
	==================
	Name: Tanzim
	NID: 18101326
	Nationality: Japanese
	Skills: []
	------------------ 	
	Skills: Mat110, PHY111, ENG101
	==================
	Name: Tanzim
	NID: 18101326
	Nationality: Japanese
	Skills: ['Mat110', 'PHY111', 'ENG101']
	==================
	[Bangladeshi(Rahim), Japanese(Tanzim)]
	[Bangladeshi(Rahim)]
	[Japanese(Tanzim)]